An Explainable Rule-Based Incident Response System Using Forward Chaining and Heuristic Prioritization

Nila Ela
Department of Computer Science and Engineering (Cyber Security)
Rajalakshmi Engineering College
Chennai, India

Sreedharshini M
Department of Computer Science and Engineering (Cyber Security)
Rajalakshmi Engineering College
Chennai, India

***Abstract*—Security operations teams must triage large volumes of alerts from brute-force, port-scan, malware, SQL-injection, phishing and denial-of-service activity, and manual triage is slow and varies between analysts. This paper presents an incident response system that classifies each reported incident, assigns it a priority score from 0 to 100 and recommends response actions, while keeping every decision traceable. The analysis engine combines a rule-based expert system with forward-chaining inference, a heuristic priority function over asset criticality, attack frequency and impact, and constraint-based selection of response actions. The system is implemented as a Spring Boot REST service with a MySQL store and a web dashboard for monitoring, reporting and attack-origin visualization. In a preliminary evaluation on nine incidents across four attack classes, the system rated all four brute-force attempts on an administrator account as Critical and the other five incidents as Medium, with a mean priority score of 47.5, and all create, read, update and delete operations passed API testing. Because the approach needs no labelled training data and exposes the rules behind each result, it suits teams that require transparent, repeatable triage.**

***Index Terms*—incident response, expert systems, forward chaining, heuristic evaluation, constraint satisfaction, security operations.**

I. Introduction

Every network service an organization exposes produces logs and alerts. A security operations team has to decide which alerts are real incidents, which incident to handle first, and what to do about it. The NIST incident handling guide [1] places this work in its detection and analysis phase, where analysts validate, document and prioritize incidents before containment begins. In practice that step is done by hand: an analyst reads each alert, judges its severity from experience, and chooses a response, so the outcome depends on who is on shift and critical incidents can wait in the same queue as harmless noise.

Research on automating security analysis has concentrated on detection. Surveys of data-mining and machine-learning (ML) intrusion detection [2], [3] report a wide range of methods, but Sommer and Paxson [4] show that ML detectors face a high cost of errors, highly variable traffic and a semantic gap between an alert and its operational meaning. These methods also need labelled datasets, and their output usually ends at an alert, leaving prioritization and response to people.

Knowledge-based artificial intelligence takes a different route: expert knowledge is written as rules, applied in the same way to every case, and each conclusion can be traced to the rules that produced it [5]. This paper applies that approach to the stage after detection. Its contributions are:

1) A four-stage analysis pipeline that classifies an incident with production rules, derives severity by forward chaining with a recorded rule trace, scores priority with a heuristic function, and selects response actions under constraints.
2) An implementation as a layered Spring Boot and MySQL REST service with a web dashboard covering six attack classes, incident reporting, monitoring, analytics and CSV export.
3) A preliminary evaluation on nine incidents with functional API testing, and a threat model for the response system itself.

Section II reviews related work, Section III describes the architecture, Section IV the analysis method, Section V the implementation, Section VI the results, Section VII the security of the system, and Section VIII concludes.

II. Related Work

A. Intrusion Detection

Denning's intrusion-detection model [6] introduced statistical profiles of normal behaviour for spotting anomalies. Signature-based systems such as Snort [7] match traffic against rules for known attacks and remain common because their alerts are easy to interpret. The surveys in [2] and [3] cover ML and data-mining detectors and note that no single technique suits every attack and that results depend on dataset choice. These systems produce alerts; they do not rank incidents or recommend responses.

B. Rule-Based Expert Systems

Production-rule systems represent knowledge as condition-action pairs and reason by forward chaining from known facts to conclusions [5]. The Rete algorithm [8] made rule matching efficient for large rule bases. Their main advantage for security operations is explainability: the fired rules form a readable justification for each decision.

C. Scoring and Prioritization

The Common Vulnerability Scoring System [9] shows that a fixed formula over a few metrics can produce consistent severity scores across organizations, although it scores vulnerabilities rather than live incidents. Heuristic search, as in A* [10], orders alternatives with an evaluation function instead of exhaustive analysis. Our priority function follows the same idea for incidents. Table I compares the approaches.

TABLE I
COMPARISON OF INCIDENT HANDLING APPROACHES

Approach	Labelled data	Explainable	Prioritizes	Recommends
Signature IDS [7]	No	Partly	No	No
ML-based IDS [2], [3]	Yes	Low	No	No
Manual triage [1]	No	Analyst-dependent	Manual	Manual
Proposed	**No**	**Rule trace**	**0-100 score**	**Yes**

III. System Architecture

The system is organized in four layers (Fig. 1). The presentation layer is a web front end used by the security analyst. The application layer is a Spring Boot REST service split into controller, service, repository and entity classes. The AI analysis engine contains the rule base, the forward-chaining inference engine, the heuristic evaluator and the constraint-based response selector. The data layer is a MySQL database whose schema follows six classes: User, Role, Incident, IncidentAnalysis, ResponseAction and SystemLog. One user creates many incidents, each incident has one analysis, and each analysis recommends one or more response actions.

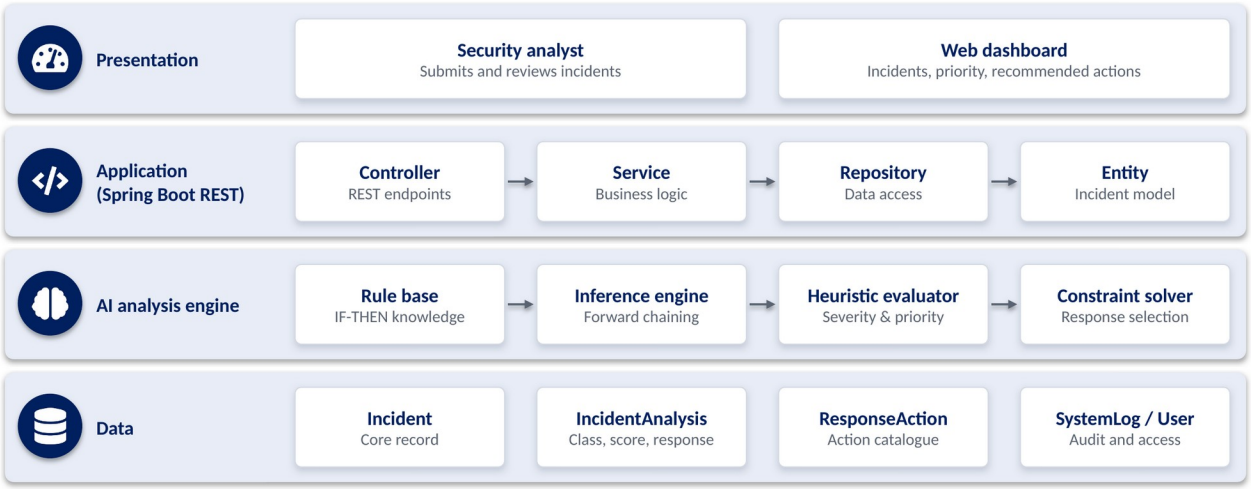


Fig. 1. Layered architecture of the proposed incident response system.

An incident moves through five stages: input, preprocessing, AI analysis, evaluation and response output. The result is written to MySQL and returned to the dashboard through the REST interface.

IV. Methodology

A. Incident Representation

An incident is the tuple $x = (s, a, n, t)$, where s is the source IPv4 address, a is the attack class from the set {Brute Force, SQL Injection, Malware, DDoS, Port Scan, Phishing}, n is the number of failed attempts and t is the target asset. Preprocessing checks that a is present, that s is a valid address and that n is a non-negative integer, then converts the record into an initial fact set F_0 containing attack(a), source(s), attempts(n), target(t) and keyword facts for high-value terms such as admin, database and root found in t.

B. Rule Base and Forward Chaining

Each rule r has a condition set C_r and a conclusion c_r. Rules are grouped by attack class so that each class is judged by its own evidence. Representative rules appear in Table II. Forward chaining (Algorithm 1) repeatedly fires every rule whose conditions hold in the current fact set and adds its conclusion, stopping when no new fact is derived. Because the set of derivable facts is finite, the procedure terminates; with naive matching each pass costs $O(|R| \cdot |F|)$, which Rete-style matching [8] can reduce for larger rule bases. The list of fired rules T is stored with the analysis as the explanation shown to the analyst.

TABLE II
REPRESENTATIVE RULE STRUCTURE

Rule	IF	THEN
R1	attack = Brute Force ∧ attempts above threshold	brute-force confirmed
R2	brute-force confirmed ∧ target has admin or root	severity = Critical
R3	attack = Malware ∧ target has database	high-value asset

Rule	IF	THEN
R4	attack = DDoS ∧ public service	availability impact
R5	severity ∈ {High, Critical}	notify analyst

Algorithm 1: Forward chaining with prioritization

```
Input: fact set F, rule base R, constraints K
T <- empty list
repeat
  N <- {}
  for each r in R do
    if C_r is a subset of F and c_r not in F then
      N <- N + {c_r};  append r to T
  F <- F + N
until N is empty
P <- HEURISTIC(F)              // Eq. (1)
S <- LEVEL(P, F)               // Eq. (2)
R* <- {act in A_c : K holds}   // Eq. (3)
return (F, T, P, S, R*)
```

C. Heuristic Priority Function

Priority is computed from three factors, each normalized to [0, 1]: asset criticality $C(t)$, derived from the target type and keywords; attack frequency $\Phi(n, s)$, derived from failed attempts and repeated sources; and attack impact $I(a)$, fixed per attack class. The score is

$$P = 100 \cdot (w_c\, C(t) + w_f\, \Phi(n, s) + w_i\, I(a)) \quad (1)$$

with non-negative weights summing to one, so P lies in [0, 100]. The severity level is obtained from thresholds $\theta_1 > \theta_2 > \theta_3$, with a keyword floor that lifts incidents on admin, database or root targets:

$$S = \begin{cases} \text{Critical}, & \text{if } P \geq \theta_1 \text{ or R2 fired} \\ \text{High}, & \text{if } \theta_2 \leq P < \theta_1 \\ \text{Medium}, & \text{if } \theta_3 \leq P < \theta_2 \\ \text{Low}, & \text{otherwise} \end{cases} \quad (2)$$

Like the evaluation function in A* [10], the score orders incidents without exhaustive analysis of every outcome, and its inputs are visible to the analyst.

D. Constraint-Based Response Selection

For the derived attack class c the system holds a candidate action set A_c, for example blocking a source address, locking and resetting an account, rate-limiting a service or isolating a host. Constraints K encode conditions such as never auto-blocking a whitelisted internal host and requiring analyst escalation for Critical incidents. The recommended set is

$$R^* = \{\, \alpha \in A_c : k(\alpha, F) \text{ holds for every } k \in K \,\} \quad (3)$$

so the analyst receives only actions valid for that incident and asset.

V. Implementation

The backend is written in Java with Spring Boot. The Incident entity is persisted through a repository, a service class holds the business logic and the analysis pipeline, and a controller exposes create, read, update and delete (CRUD) operations as REST endpoints exchanging JSON. MySQL stores incidents, analysis results, response actions, users and logs. Endpoints were tested with Postman.

The web front end provides user sign-in and account creation, a reporting form (Fig. 2) for the four input attributes with a panel describing the three analysis stages, a searchable incident list with severity and status filters, a dashboard with summary cards, a seven-day trend, severity and attack-type charts, a list of open critical incidents and a 3D globe that plots geolocated public source addresses, and a reports page with 7-day, 30-day and all-time windows and CSV export (Fig. 3). A simulate-attack control creates sample incidents for testing.

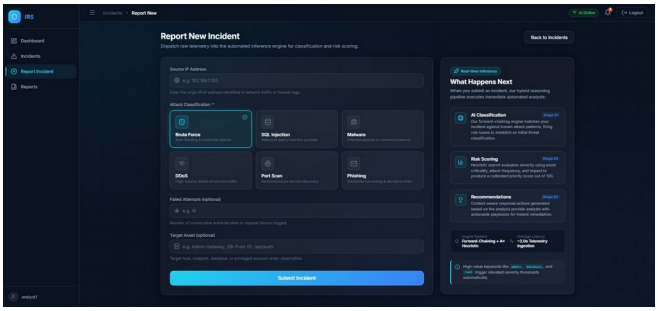


Fig. 2. Incident reporting form with the six attack classes and the analysis stages.



Fig. 3. Reports and analytics page with incident trend, attack categories, severity and response status.

VI. Results and Discussion

A. Functional Testing

All four CRUD operations behaved as expected in Postman (Table III): created incidents were stored in MySQL, returned on read, updated and removed.

TABLE III
API TEST CASES

Test	Operation	Expected outcome	Result
TC1	POST	Incident stored	Pass
TC2	GET	Incident returned	Pass
TC3	PUT	Fields updated	Pass
TC4	DELETE	Incident removed	Pass

B. Incident Analysis

Nine incidents were recorded between 3 and 9 October 2026 (Table IV, Fig. 4). The four brute-force incidents from 10.0.0.5 against an administrator account were rated Critical, which matches rule R2 for privileged targets. Three malware incidents against a database server, one DDoS against a public website and one port scan of an office router were rated Medium. The mean priority score was 47.5 out of 100.

TABLE IV
INCIDENTS IN THE TEST SET

ID	Source	Attack	Target	Severity	Priority
1-3	10.0.0.5	Brute Force	Admin account	Critical	N/A
4	10.0.0.5	Brute Force	Admin account	Critical	Critical
5-7	192.168.1.50	Malware	Database server	Medium	Medium
8	208.67.222.222	DDoS	Public website	Medium	Medium
9	9.9.9.9	Port Scan	Office router	Medium	Medium

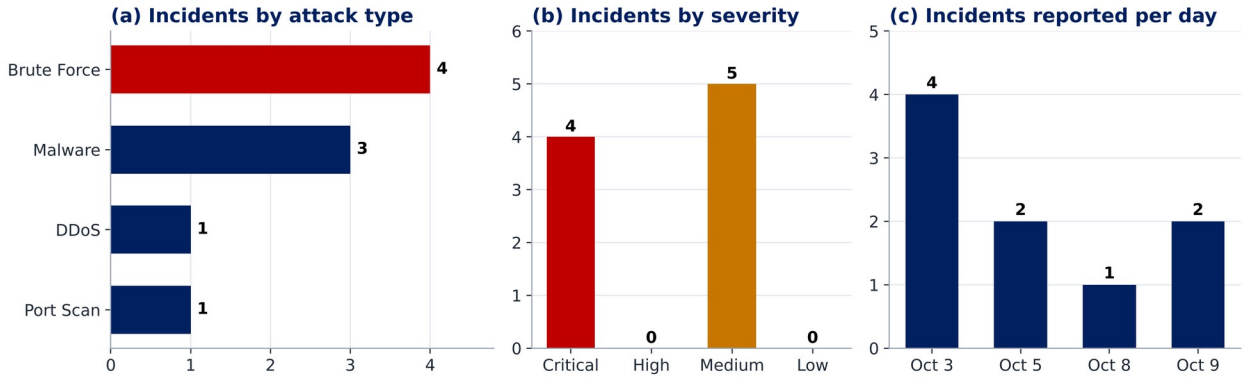


Fig. 4. Distribution of the nine test incidents by (a) attack type, (b) severity and (c) day of reporting.

Two observations need follow-up. Incidents 1 to 3 have the same attributes as incident 4 but carry no priority value; they were stored before priority scoring was applied and were not re-scored, which shows the need for a re-analysis operation on existing records. The malware incidents on the database server stayed at Medium even though the target contains a high-value keyword, so the keyword floor for malware on database assets should be reviewed with analysts during threshold calibration. On the dashboard globe, one of the nine source addresses was plotted, seven were private or local and therefore not geolocated, and one could not be resolved.

C. Comparison with Manual Triage

In the manual workflow each of the nine incidents would require an analyst to read, rank and answer it. In the proposed system the class, severity, priority and recommended actions are produced when the incident is submitted, the four Critical incidents appear first in the list of open critical incidents, and the fired rules explain each rating. The same facts always yield the same result, which removes the shift-to-shift variation of manual ranking.

D. Limitations

The evaluation is preliminary: nine incidents, no independent ground-truth labels and no accuracy, precision or recall figures. Weights and thresholds in (1) and (2) are set by hand. Incidents are entered manually rather than ingested from sensors, and the resolution workflow was not exercised, so the resolution rate in the test period was 0%.

VII. Security of the Response System

Because the system stores sensitive incident data, it is itself a target. The attack surfaces are the dashboard, REST APIs, authentication system, AI analysis module and database. Threats include unauthorized access, SQL injection, API abuse, data manipulation, credential theft, denial-of-service and tampering with the rule base. Controls are sign-in with role-based authorization, input validation and parameterized queries, rejection of unauthorized API requests, an audit trail in SystemLog, a controlled rule set with validated inputs, and error handling, backups and request limits for availability.

VIII. Conclusion and Future Work

This paper presented an explainable incident response system that classifies incidents with production rules, derives severity by forward chaining, scores priority with a heuristic function and recommends responses under constraints. The implementation on Spring Boot and MySQL with a web dashboard separated the four critical brute-force attacks on an administrator account from five lower-risk incidents at the moment of submission, without labelled training data and with a rule trace for every decision.

Future work will evaluate classification accuracy and priority consistency on labelled scenarios for all six attack classes, calibrate the weights and thresholds with analysts, add re-analysis of stored incidents, complete the resolution workflow, and ingest incidents directly from firewall and IDS logs.

ACKNOWLEDGMENT

The authors thank Dr. P. Tamilselvi, Professor and Head, Department of Computer Science and Engineering (Cyber Security), Rajalakshmi Engineering College, for her guidance and support.

REFERENCES

[1] P. Cichonski, T. Millar, T. Grance, and K. Scarfone, “Computer security incident handling guide,” *NIST Special Publication 800-61 Rev. 2*, Aug. 2012.

[2] A. L. Buczak and E. Guven, “A survey of data mining and machine learning methods for cyber security intrusion detection,” *IEEE Commun. Surveys Tuts.*, vol. 18, no. 2, pp. 1153-1176, 2016.

[3] A. Khraisat, I. Gondal, P. Vamplew, and J. Kamruzzaman, “Survey of intrusion detection systems: Techniques, datasets and challenges,” *Cybersecurity*, vol. 2, art. 20, 2019.

[4] R. Sommer and V. Paxson, “Outside the closed world: On using machine learning for network intrusion detection,” in *Proc. IEEE Symp. Security and Privacy*, 2010, pp. 305-316.

[5] S. J. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Hoboken, NJ, USA: Pearson, 2020.

[6] D. E. Denning, “An intrusion-detection model,” *IEEE Trans. Softw. Eng.*, vol. SE-13, no. 2, pp. 222-232, Feb. 1987.

[7] M. Roesch, “Snort: Lightweight intrusion detection for networks,” in *Proc. 13th USENIX Conf. System Administration (LISA)*, 1999, pp. 229-238.

[8] C. L. Forgy, “Rete: A fast algorithm for the many pattern/many object pattern match problem,” *Artificial Intelligence*, vol. 19, no. 1, pp. 17-37, 1982.

[9] P. Mell, K. Scarfone, and S. Romanosky, “A complete guide to the Common Vulnerability Scoring System version 2.0,” *FIRST*, Jun. 2007.

[10] P. E. Hart, N. J. Nilsson, and B. Raphael, “A formal basis for the heuristic determination of minimum cost paths,” *IEEE Trans. Syst. Sci. Cybern.*, vol. 4, no. 2, pp. 100-107, Jul. 1968.